Final Report

Aaron Brady, Isaac Vissers, Jason Gillanders

Abstract

This document presents our design of a Secure File System (SFS) that demonstrates our knowledge of the lecture content from ECE 422. The motivation behind the system is to allow internal users to store data on an untrusted file server. The system supports an admin user, group management, authenticated access, encrypted file and directory storage, and permission controls. The system is implemented as a monolithic Python application that supports an interactive command-line interface and provides secure and reliable file management within a single machine environment.

Introduction

The objective of this project is to design and implement a Secure File System (SFS) that enables internal users to securely store data on an untrusted file server. Although external users may be able to observe that files and directories exist, they must not be able to read file names, access file contents, or modify stored data without detection. The system therefore enforces authentication, encrypted storage, controlled file sharing, and integrity verification to protect user data.

The design of the SFS is guided by the principles of the CIA triad. Confidentiality is achieved by encrypting all file and directory names and contents on disk, ensuring that only authenticated and authorized users can access protected resources. Integrity is enforced through cryptographic mechanisms that detect tampering or corruption of files and metadata, with users notified of any integrity violations upon login. Availability is maintained by providing authenticated users reliable access to their files and directories through a structured permission model and an interactive command-line interface.

The system supports multiple users and groups, and controlled file sharing through permissions. Implemented as a monolithic Python application with an interactive CLI, the Secure File System demonstrates how core security principles can be applied to file system design while operating within an untrusted single-machine environment.

Detailed Design

Authentication

The secure file system uses knowledge based authentication with a username and password combination for authenticating users. User details used for authentication are stored in encrypted metadata files.

To initialize the authentication process, we created a script which generates a random password salt (Key Salt), which is used in conjunction with the username and password to create the AES key to decrypt the

user metadata file. This salt value is generated once and is constant. This script also creates an “admin” user which is responsible for creating and managing users and groups.

To create a user, the “admin” user must be logged in, and the user provides a username and password. The system then creates a random static 16 byte salt value (Individual Password Salt) for each new user which is stored in the encrypted metadata file. The system then uses Argon2 hashing of the (entered password + Individual Password Salt) to create a static verification hash that is stored in the user metadata file. The plaintext password is never saved within the user metadata file or in the system. The user metadata file is then encrypted and saved on disk as a nonce and ciphertext with the filename being the SHA256 hash of the username.

To authenticate a user allowing them to log in to the system, a user must enter a valid username and password. The system then generates the user metadata filename by SHA256 hashing the username, and the user metadata file AES key, which is the username+password+Key Salt. The system then decrypts the file using the decryption key, and as a final authentication verification process, it hashes the entered password + Individual Password Salt which is compared with the verification hash.

Encryption

For the encryption methods, we chose to use a symmetric encryption approach to secure our file system. By using a single shared key for file and group metadata, the system ensures that only users with the correct key can decrypt and access the files, maintaining confidentiality. We selected to use AES encryption as it is an industry standard and is well supported by the libraries we chose to use for this project.

To encrypt the user, group, and file/directory metadata files in our system, we utilized symmetric encryption through the use of AES encryption to ensure confidentiality and integrity. Each encrypted user metadata file holds paths to the files and groups that a specific user owns and is a member of, and the key associated with the specific metadata files.

On user creation, the system prompts the user for a username and password. The system has a random static 12 byte salt value that will be used for both password verification and encryption purposes. The user key is created which is a SHA256 hash of the combination of the username + password + random salt. Before the system saves the user metadata file to the system, it converts the user data to json format and then we encrypt the data. To encrypt the file, for each user metadata save, a new random 12 byte nonce is created. We then use AES encryption to encrypt the json data using the nonce and user key to create a cipher text of the user data. The system then will create the user metadata file by converting the nonce and cipher text to bytes and saving. All communication between the system and the files are encrypted. This results in an hidden AES encrypted file locked with the individual user key, in a non-readable byte state named as a SHA256 hash of the username.

On user login, the user key is created by hashing the entered username + entered password + random salt. This key will then be used to attempt to decrypt the user specific metadata file. The system will read in the byte format of the file nonce and cipher text ensuring that all communication between the file and system stays encrypted. The system will then use the nonce and user key to attempt to decrypt the cipher

text through AES. If the user is successful in decryption, the user data will be loaded into the system, and if the file is not decrypted, then an error will be raised.

For group encryption, when a group is created, we create a group master key which is a 32 byte AES key. The group metadata is converted to json format within the system, and a new 12 byte nonce value is created for each save. The group json is converted to an AES encrypted cipher text. The system then writes the nonce and cipher text in byte form to the file, ensuring that communication between the system and file is encrypted. This results in a hidden locked group meta data file locked with the group key, that is not human readable.

When a user is added to a group, the system loads the user's encrypted data into the system, decrypts the user file, and adds both a path to the hidden encrypted group file and group key to the user file. The user file is then encrypted and sent to the user file. The user file will contain all keys to the groups that they are members of. Since the user meta data files are encrypted, the group key remains protected at rest.

For file encryption, when a file is created, a random 32 byte AES key is generated. The file metadata is converted into json format within the system, and similar to groups, a new 12 byte nonce value is created for each save. The data is then encrypted into an AES cipher text of the nonce and json body that is encrypted using the AES file key. The file will then create an integrity hash that is used to determine integrity of the file. The integrity hash is created by hashing the nonce and encrypted cipher text. The nonce, encrypted cipher text and integrity hash are then saved to the file metadata file, and are communicated encrypted from the system to the file. The user that is creating the file will save a path to the file, and the AES key into their user metadata file.

When a user logs into the system, they will check the files that they own, by hashing the nonce and cipher text and comparing with the integrity hash. Because the user metadata file contains the file path, the system is able to determine the file that was tampered with, and indicates the file owner.

Access Permissions

The secure file system allows users to set individual access permissions for each file and directory that they are the owner of. These permissions are user, group, and all.

The ‘user’ permission restricts file access to only the owner of the file. That is, only the owner of a given file can access, read, write, and do other operations on that file. Other users, even within the same group, cannot read or write these files. In addition, other users in general do not even know that these files exist. That is, if User 1 and User 2 are not in the same group, all of User 1’s files that are set with the ‘user’ permission are essentially invisible to User 2; they cannot even be found using CLI commands such as ls and cd. However, if User 1 and User 2 are in the same group, then User 2 can see that User 1 has private files, although their names are hashed (so User 2 does not know what files they are), and they cannot be read or written to.

The ‘group’ permission allows users to give read/write permissions to users within their own group. That is, users that are in the same group can access directories (use cd, pwd, and ls within the directory) and files (read/write) with the ‘group’ permission. Files that are within a directory with a ‘group’ permission are shown with their encrypted names to users within the group. For example, let’s say User 1 and User 2 are in the same group. User 2 can navigate to User 1’s directory, since it automatically has the ‘group’ permission. User 2 can see User 1’s files exist, but with their encrypted names. Any other files that User 1 has with the ‘group’ permission, User 2 can navigate to and read/write them.

The ‘all’ permission is very similar to the ‘group’ permission, except that it applies to all users of the SFS. It essentially uses a group called ‘all’ that all users of the SFS are a member of, so that all users can read/write the files and directories with the ‘all’ permission.

This access permission system is an implementation of Discretionary Access Control. This is because only the owner of each resource can change the permissions of that resource. The permission of a given file is written to that file’s metadata file. A file metadata file contains the file name, the owner of the file, the permission of the file, the encrypted name of the file, the body of the file, the encrypted file key, and the path to the file. This file metadata file is encrypted using AES, where the symmetric file key is stored in both the owner’s user metadata file, as well as any group metadata file that the owner is a member of.

We implemented this access permissions system through the use of user and group metadata files. The user metadata file includes the username, the actual path using encrypted names to the metadata file, a list of the file keys for the files that the user is the owner of, other information (such as decrypted file name) for each owned file key, a list of all user keys of the SFS (this is used by the admin user only), and a list of the group keys for the groups that this user is a member of. The group metadata file includes the group name, the encrypted name of the group, the members of the group, the files of the members of the group with the ‘group’ permission, and the path to the group. These metadata files are encrypted using AES, where the group symmetric encryption key is stored within the user’s metadata file, and the user encryption key is calculated at login. When a file’s permission is changed to ‘group’ or ‘all’ from ‘user’, the associated file key is added to the relevant group metadata file, so that other members of the group can now access the file freely.

Within the CLI commands that we had to implement (such as cd, ls, echo, cat, etc), we added in a check to see whether these file keys were in any metadata files (user/group) that the user has access to. If it found the file keys, then the system knows that the user has access to those said files. If a file’s file key was not found in the accessed metadata files, then the user will not have access to the file.

We also restricted certain CLI commands (such as creating files, creating directories, renaming files, and setting permissions) to be used by the file/directory owner only. This is determined by checking if a user is within their own home directory or not. If they are, they can freely use these commands. If not, these commands are not allowed.

Technologies

We implemented this program in Python because it simplifies the development process. We all have experience with Python, and it has well maintained libraries for implementing CLI and encryption.

- Library for interactive shell
    - Cmd
        - Built in library
        - Well documented
        - This is the exact use case for this library
    - Cryptography (https://pypi.org/project/cryptography/)
        - Well documented
        - Implements AES symmetrical encryption
        - Implemented Argon2 for password hashing
        - Implements SHA256 for hashing and integrity

Methodologies

We used Agile and Test Driven Development methodologies for this project.

We believe that Agile is well suited for this project as the implementation can be broken down into chunks which can be implemented in an iterative manner. (e.g., authentication, encryption, cli, user operations, file access options, tamper detection). Implementing these in iterations allows us to have working code early, validate our code early, and address issues early. We especially think that this is a good methodology because we have never implemented encryption before and we want the ability to change our requirements, design and implementation if that is necessary.

We believe that test driven development will be effective for this project because correctness, security, and reliability of the system are crucial and this allows us to cover edge cases, ensure the validity of the encryption algorithm we implement, and that the system fails safety.

Tools

We used GitHub for version control with GitHub actions for our CI/CD pipeline. In our CI pipeline we use pytest to run our unit tests, ruff for code linting and formatting. For our CD portion of the pipeline (release on tag) we want to generate the artifacts required for a user to run our project, we combine the python code and libraries into a single executable that can be ran without directly installing independencies, as well as a readme explaining how to use the program which will be released via github releases.

We chose this toolchain because it is lightweight, well-supported, and integrates seamlessly with our development workflow for the purposes of a demo. GitHub Actions provides automated validation of code quality and correctness, which is especially important for a security-focused system where regressions could introduce vulnerabilities. The release-on-tag strategy ensures that only tested and stable versions are distributed, promoting reproducibility and reliability for users and TAs who will be present for our demo.

User Stories

- **US-01:** As an administrator, I want to create user accounts so that authorized individuals can access the Secure File System.
  Acceptance Criteria: An authenticated administrator can create a new user account with a unique username and password, after which the user has a provisioned home directory and can successfully log in, with the password stored securely and not in plaintext.
- **US-02:** As an administrator, I want to create groups so that users can be organized under shared access boundaries.
  Acceptance Criteria: An authenticated administrator can create a new group that is stored in the system and available for assigning users.
- **US-03:** As an administrator, I want to add users to groups so that I can manage group-based access control.
  Acceptance Criteria: An authenticated administrator can add an existing user to an existing group, after which the user inherits that group's access permissions.
- **US-04:** As an administrator, I want to remove users from groups so that I can revoke their group-based access.
  Acceptance Criteria: An authenticated administrator can remove a user from a group, after which the user no longer has access to group-protected resources.
- **US-05:** As an administrator, I want to view the usernames of all registered users so that I can manage the system, but I should not be able to view user passwords.
  Acceptance Criteria: An authenticated administrator can view a list of all registered usernames, but cannot view or retrieve any user passwords.
- **US-06:** As a user, I want to securely authenticate to the system so that only authorized users can access files.
  Acceptance Criteria: A user can log in using valid credentials, and access to the system is denied if authentication fails.
- **US-07:** As a user, I want an automatically provisioned home directory so that I have a private workspace.
  Acceptance Criteria: Upon successful account creation, the system automatically creates a home directory accessible only according to the defined permission model.
- **US-08:** As a user, I want to create files so that I can store data on the system.
  Acceptance Criteria: An authenticated user can create a file within an authorized directory, and the file is stored encrypted on disk.
- **US-09:** As a user, I want to read files I have permission to access so that I can retrieve stored data.
  Acceptance Criteria: An authenticated user can read the contents of a file only if permitted by its access controls, and the correct decrypted content is returned.
- **US-10:** As a user, I want to write to files I have permission to modify so that I can update stored data.
  Acceptance Criteria: An authenticated user can modify a file only if permitted by its access controls, and the updated content is stored encrypted on disk.

- **US-11:** As a user, I want to rename files so that I can organize my workspace.
  Acceptance Criteria: An authenticated user can rename a file they have permission to modify, and the updated filename remains encrypted on disk.
- **US-12:** As a user, I want to delete files I own or have permission to remove so that I can manage my storage.
  Acceptance Criteria: An authenticated user can delete a file only if permitted by its access controls, and the encrypted file data is removed from disk.
- **US-13:** As a user, I want to create and navigate directories within my home directory so that I can manage files hierarchically.
  Acceptance Criteria: An authenticated user can create and navigate directories within permitted locations, and directory names are stored encrypted on disk.
- **US-14:** As a file owner, I want to set permissions (user/group/all) on files and directories so that I can control who can read, write, or access them.
  Acceptance Criteria: A file owner can set user/group/all permissions on a file or directory, and those permissions are enforced by the system.
- **US-15:** As a user, I want group-based access controls enforced so that I can see files of users in my group (with encrypted names unless I have access), and not see files of users outside my group.
  Acceptance Criteria: A user can view the existence of files belonging to users in the same group according to permission rules, while files of users outside their groups are not visible.
- **US-16:** As a user, I want all files and directories stored encrypted on disk so that no one can access them without authenticating.
  Acceptance Criteria: All file and directory data stored on disk is encrypted such that it cannot be read without successful authentication through the SFS.
- **US-17:** As a user, I want file and directory names and contents encrypted at rest so that external users cannot read them directly from disk.
  Acceptance Criteria: File and directory names and contents appear as encrypted data when viewed directly on disk outside the SFS application.
- **US-18:** As a user, I want the system to detect tampering or corruption of file or directory names and contents so that integrity violations are identified.
  Acceptance Criteria: If any file or directory data is modified outside the SFS, the system detects the integrity violation during verification.
- **US-19:** As a user, I want to be notified of any integrity failures upon login so that I immediately know if my data was modified.
  Acceptance Criteria: Upon login, the system checks for integrity violations and notifies the user if any corruption or tampering is detected.

Deployment Instructions / Setup

Our project uses Github Actions for CI/CD with a clear split on every push. Our CI ensures code quality on every push by running all of our unit/integration tests, and running a linter. All new versions are tagged using semantic versioning [1] (for example v1.0.0). When a tag is pushed our CD builds native binaries for Linux, macOS and windows and publishes them as a GitHub release where they can be downloaded as

individual artifacts. This is continuous delivery (automated packaging and release) but not automatic deployment (as installation and setup is still a manual process).

If a user wants to use our file system they need to go to github releases for our project and install the latest prebuilt binaries for their OS. There are 2 binaries needed: secure-fs-admin-setup-<os>-vx.x.x.<ext> and secure-fs-<os>-vx.x.x.<ext>. After downloading, the user should run the admin setup binary. This will create an admin with the credentials (admin/admin) and the required file system structure setup. From the same parent directory the user can then run the main secure file system binary in order to access the system. All required system documents will be created in a storage/subdirectory. So the binaries should be run in a writable folder.

User Guide

- Go to the github releases pages https://github.com/isaacvissers/secure_file_system/releases
    - Download the 2 binaries for your OS
        - secure-fs-<OS>-x86_64-vX.X.X.<extension>
        - secure-fs-admin-setup-<OS>-x86_64-vX.X.X.<extension>
- Unzip both folders and place the binary executables in the same directory. It must be a writable directory
- Run the admin setup, which creates an admin user with the credentials (user: admin, pass: admin). This admin user can be used to create and manage users and groups
- Run the main program. It will launch a CLI to use the program. The CLI Provides the following commands
    - Available to all users
        - `help`
            - Print all available commands
        - `login`
            - Authenticate the user or admin in with username and password
        - `logout`
            - Log the current user out
        - `exit`
            - Close the application
    - Admin Only
        - `create_user`
            - create a new user with specified username and password
            - This will also create a home directory for the user and add the user to the all group
        - `create_group`
            - Create a new group
        - `add_user_to_group`
            - add specified user to the specified group
            - This will also add any of that users files with group permissions to that group

        - `remove_user_from_group`
            - Remove specified user from the specified group
            - This will also remove the users files from that group
    - User Only
        - `cat <file_name>`
            - Print the contents of <file_name> as plaintext if you have access to the file
        - `cd <directory_name>`
            - Traverse into <directory_name>
            - This command only supports relative paths
            - `cd ..` can be used to return to the parent directory
            - `cd` can be used to return to the user's home directory
            - We only support traversing into a directory you have access to
        - `Echo`
            - `echo "TEXT" > <file_name>`
                - Sets the contents of <file_name> to TEXT
                - If <file_name> doesn't exist in the current working directory it will be created
            - `echo "TEXT" >> filename`
                - Appends "TEXT" to the end of the contents of <file_name>
                - If <file_name> doesn't exist in the current working directory it will be created
        - `get_permissions <name>`
            - Prints the permissions of file or directory <name>
        - `ls`
            - Print the contents of the current working directory
            - This will include encrypted file/directory names if you do not have access, and real decrypted names for files/directories you have access to
        - `mkdir <directory_name>`
            - Creates a subdirectory named <directory_name> in the current working directory
            - We only support creating files within the user's home directory
            - Directories are created with default permissions of `user`
        - `mv <file_name> <new_name>`
            - Rename <file_name> to <new_name>
            - We only support renaming files, not directories
            - We only support renaming files you are the owner of (within your home directory)
        - `pwd`
            - Prints the current working directory
        - `set_permissions <name> <permission>`
            - Set permissions of file or directory <name> to <permission>

        - We only support setting the permissions of files you are the owner of (within your home directory)
    - `touch <file_name>`
        - Creates <file_name> in the current working directory
        - We only support creating files within the user's home directory
        - Files are created with default permissions of `user`

Conclusion

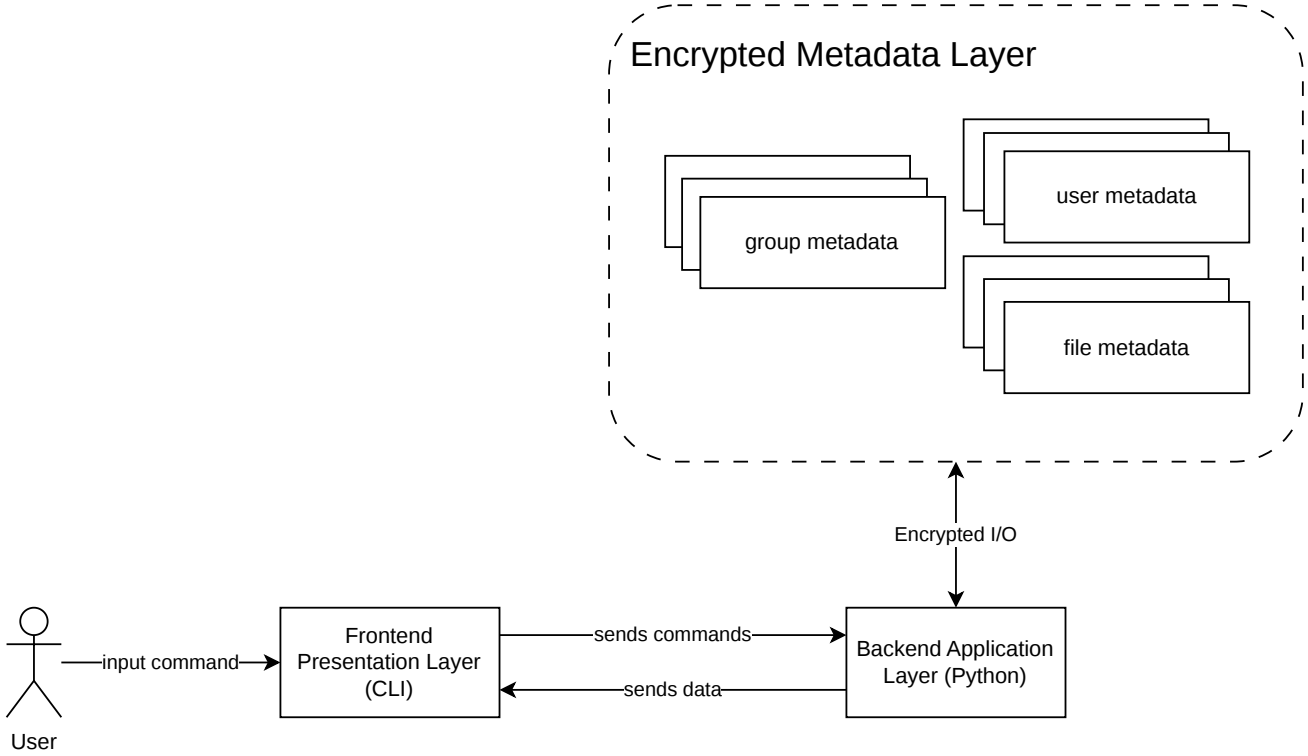
The objective of this project was to design and implement a Secure File System (SFS) that enables internal users to securely store data on an untrusted file server. We used knowledge learned in ECE 422 lectures, such as authentication, access control, integrity, and encryption, to implement this SFS. We successfully followed the CIA triad by protecting confidentiality, integrity, and availability. Confidentiality was achieved by encrypting created files/directories, their names, and all the metadata files used to implement the system, where only the authenticated and authorized users can read/write to their given files. This includes using authentication and access control (file permissions) to restrict who can access what. Integrity was achieved by storing a hash value of the file/directory, and checking that that hash value is equivalent to the calculated hash value on user login, using SHA256 hashing. Availability was achieved by using an intuitive CLI app that users can easily access.

The system supports multiple users and groups, and controlled file sharing through permissions. Implemented as a monolithic Python application with an interactive CLI, the Secure File System demonstrates how core security principles can be applied to file system design while operating within an untrusted single-machine environment.

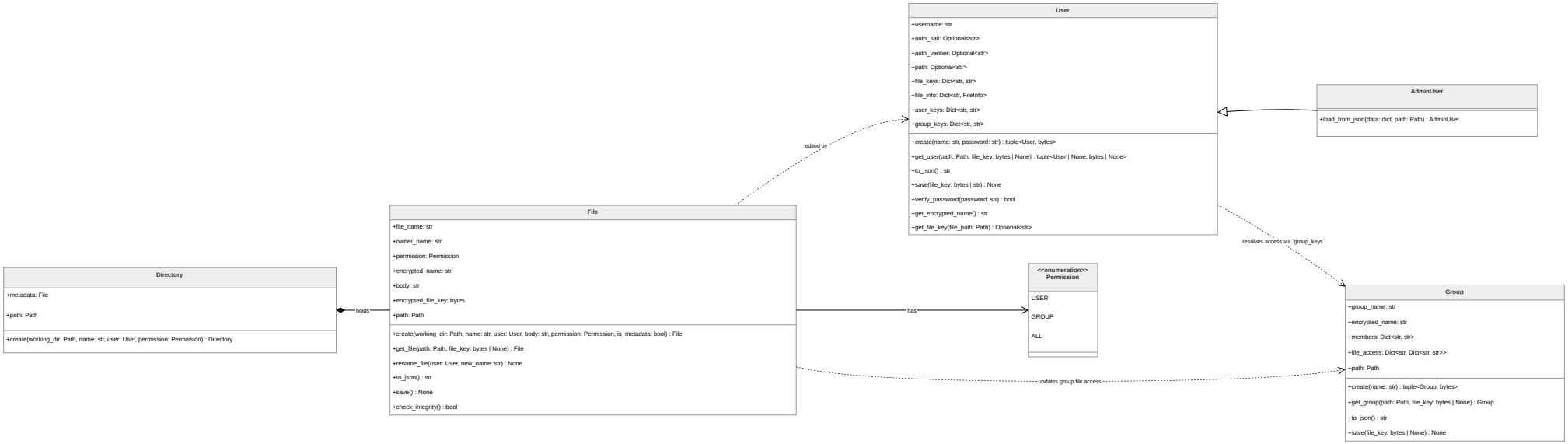
References

[1] https://semver.org
[2] https://pypi.org/project/cryptography/
[3] https://cybernews.com/resources/what-is-aes-encryption/

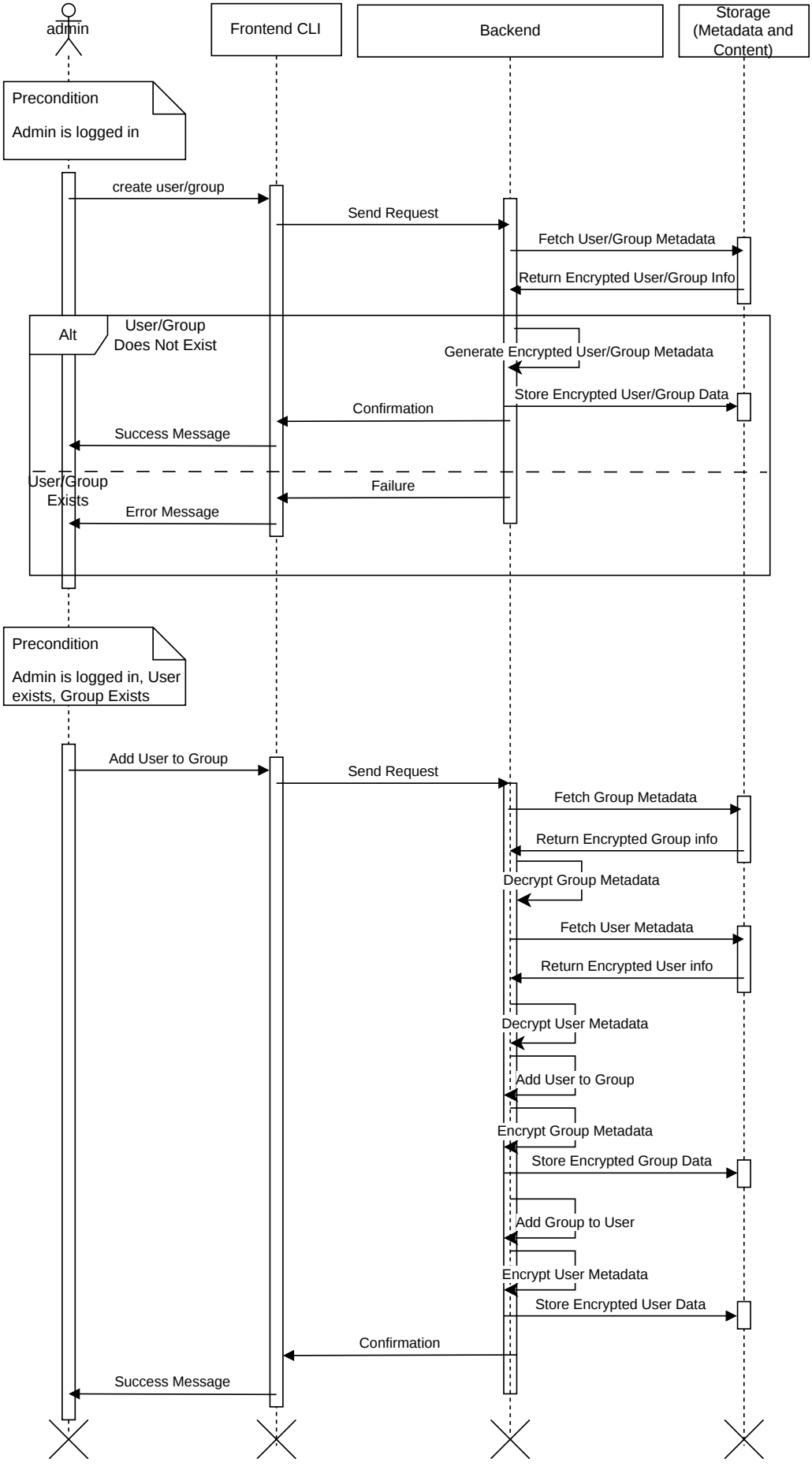
Architecture Diagram



Class Diagram



Sequence Diagram - Create User/Group



State Machine Diagram - Login